AWS LAMBDA

CONTENIDO

AWS LAMBDA 2
CÓMPUTO SIN SERVIDOR 2
¿QUÉ ES AWS LAMBDA? 3
RUNTIME DE LAMBDA 3
TRIGGERS DE LAMBDA 4
CARACTERÍSTICAS 5
FUNCIONAMIENTO 6
CUANDO USAR LAMBDA 7
BENEFICIOS 7
CASOS DE USO 8
ANATOMÍA DE UNA FUNCIÓN LAMBDA 8
RESTRICCIONES DE LAS FUNCIONES LAMBDA 9
EJEMPLOS DE FUNCIONES LAMBDA 10
TIPS 16

AWS LAMBDA

CÓMPUTO SIN SERVIDOR

Las instancias de EC2 son máquinas virtuales que puede aprovisionar con menor fricción para ponerse en marcha en AWS. EC2 es ideal para todo tipo de casos prácticos, como ejecutar servidores web básicos hasta ejecutar clases de cómputo de alto rendimiento y todo lo que hay en medio. Aunque EC2 es increíblemente flexible, fiable y escalable, dependiendo del caso práctico, podría estar buscando alternativas para su capacidad de cómputo.

EC2 requiere que configure y administre la flota de instancias a lo largo del tiempo. Cuando usa EC2, usted es responsable de aplicar parches a las instancias cuando salen nuevos paquetes de software, de configurar el escalamiento de esas instancias y, además, de asegurarse de haber diseñado las soluciones para que se alojen de una manera altamente disponible. Eso no implica tanta administración como lo habría hecho si las alojara en las instalaciones. Pero los procesos de administración aún tendrán que estar en marcha.

Tal vez se pregunte qué otros servicios ofrece AWS para cómputo que son más convenientes desde el punto de vista de la administración. Aquí es donde entra en juego el término “sin servidor”. AWS ofrece varias opciones de cómputo sin servidor. “Sin servidor” significa que no puede ver ni acceder a la infraestructura subyacente ni a las instancias que alojan la aplicación. En cambio, AWS se encarga de toda la administración del entorno subyacente desde una perspectiva de aprovisionamiento, escalado, alta disponibilidad y mantenimiento. Todo lo que debe hacer es centrarse en la aplicación y del resto se encarga a AWS.

AWS Lambda es una opción de cómputo sin servidor, es un servicio que le permite cargar el código en lo que se denomina función de Lambda. Se debe configurar un desencadenador y desde allí el servicio espera el desencadenador. Cuando se detecta el desencadenador, el código se ejecuta de forma automática en un entorno administrado, del que no debe preocuparse demasiado porque es escalable automáticamente y de alta disponibilidad. Todo el mantenimiento en el entorno en sí lo realiza AWS. Si tiene 1 o 1000 desencadenadores entrantes Lambda escalará la función para satisfacer la demanda.

Lambda está diseñado para ejecutar códigos en menos de 15 minutos, por lo que no es para procesos de larga ejecución como aprendizaje profundo (Deep learning). Es más adecuado para el procesamiento rápido como backend web, que gestiona solicitudes o un servicio de procesamiento de informes de gastos de backend en donde cada invocación tarda menos de 15 minutos en completarse.

¿QUÉ ES AWS LAMBDA?

AWS Lambda es un servicio de computación sin servidor que ejecuta código en respuesta a eventos y administra automáticamente los recursos de computación subyacentes. Estos eventos pueden ser cambios de estado o una actualización, como que un usuario coloque un artículo en un carrito de la compra en un sitio web de comercio electrónico. Puede utilizar AWS Lambda para ampliar otros servicios de AWS con lógica personalizada o crear sus propios servicios backend que funcionen a escala, rendimiento y seguridad de AWS.

AWS Lambda ejecuta de forma automática el código en respuesta a varios eventos, como las solicitudes HTTP a través de Amazon API Gateway, las modificaciones de objetos en los buckets de Amazon Simple Storage Service (Amazon S3), las actualizaciones de tablas en Amazon DynamoDB y las transiciones de estado en AWS Step Functions.

Lambda ejecuta el código en una infraestructura informática de alta disponibilidad y lleva a cabo toda la administración de los recursos informáticos. Entre otras cosas, se encarga del mantenimiento del servidor y del sistema operativo, del aprovisionamiento de capacidad y del escalado automático, de la implementación de código y de parches de seguridad, y del monitoreo y el registro del código. Lo único que tiene que hacer es proporcionar el código.

Cuando utiliza AWS Lambda, solo paga por el tiempo de cómputo que consume. Los cargos se aplican solo cuando el código se está ejecutando. También puede ejecutar código para prácticamente cualquier tipo de aplicación o servicio backend, todo sin ninguna administración.

Por ejemplo, una función de Lambda sencilla podría implicar cambiar automáticamente el tamaño de las imágenes cargadas en la nube de AWS. En este caso, la función se activa cuando se carga una nueva imagen.

RUNTIME DE LAMBDA

El runtime de AWS Lambda es el entorno específico del lenguaje que se utiliza para ejecutar el código de una función Lambda. Este entorno proporciona las bibliotecas y variables necesarias para que el código funcione correctamente. Lambda admite varios lenguajes de programación, como Python, Node.js, Java, y más.

Cada versión principal de un lenguaje tiene su propio runtime, identificado por un nombre único, como nodejs20.x o python3.121. Puedes usar los runtimes proporcionados por AWS o crear tus propios runtimes personalizados si necesitas soporte para otros lenguajes.

AWS Lambda es compatible con varios lenguajes de programación, lo que te permite elegir el que mejor se adapte a tus necesidades. Actualmente, puedes escribir funciones Lambda en los siguientes lenguajes:

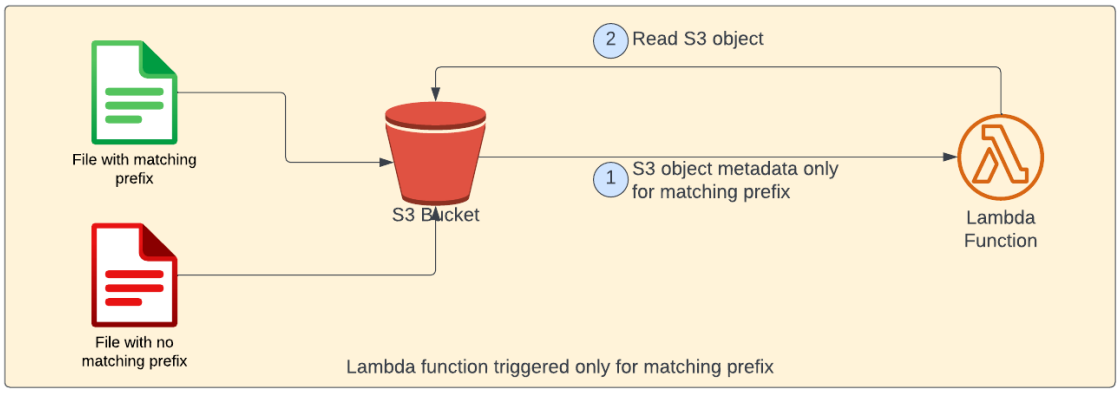
Python
Node.js
Java
C# (.NET Core)
Go
Ruby
PowerShell

Además, AWS Lambda también permite el uso de contenedores personalizados, lo que significa que puedes empaquetar tu código y sus dependencias en una imagen de Docker y ejecutarlo en Lambda, lo que te da aún más flexibilidad en cuanto a los lenguajes y entornos que puedes utilizar.

TRIGGERS DE LAMBDA

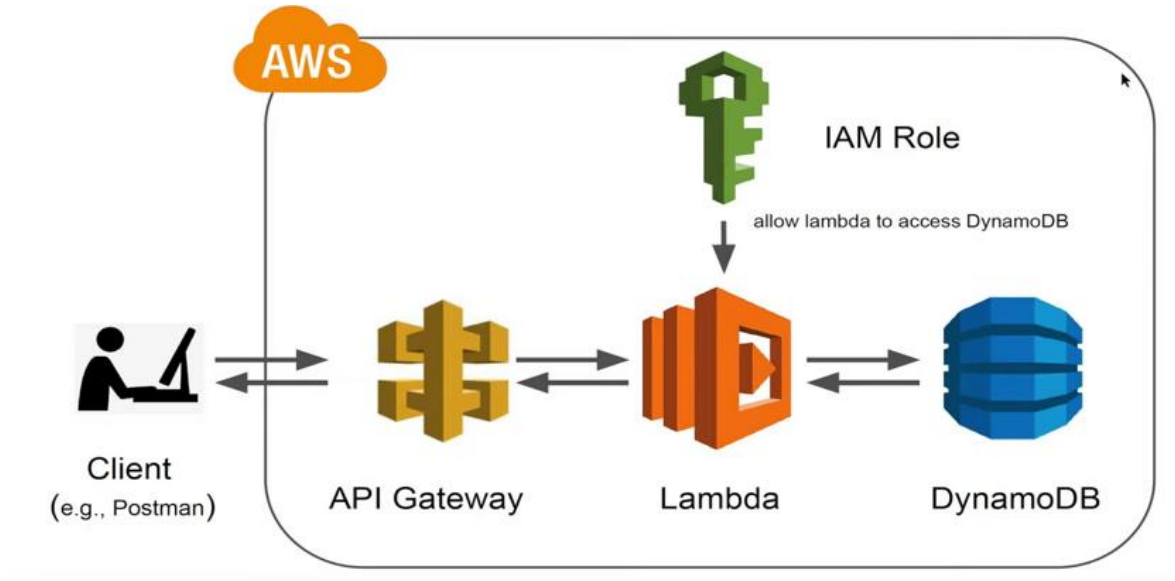
Los triggers de AWS Lambda son eventos que activan la ejecución de una función Lambda. Estos eventos pueden provenir de varios servicios de AWS o de aplicaciones externas. Algunos ejemplos comunes de triggers incluyen:

Amazon S3: Cuando se carga un archivo en un bucket de S3, puede activar una función Lambda para procesar ese archivo.



Amazon DynamoDB: Las actualizaciones en una tabla de DynamoDB pueden activar una función Lambda para procesar los cambios.

Amazon API Gateway: Las solicitudes HTTP pueden activar una función Lambda para manejar la lógica del backend de una aplicación web o móvil.



Amazon Kinesis: Los datos de streaming pueden activar una función Lambda para procesar datos en tiempo real.

Estos triggers permiten que las funciones Lambda respondan automáticamente a eventos específicos, facilitando la creación de aplicaciones sin servidor y altamente escalables.

CARACTERÍSTICAS

AWS Lambda es un servicio de computación sin servidor que ejecuta código en respuesta a eventos y administra automáticamente los recursos de computación subyacentes. Algunas de sus características principales son:

Ejecución sin servidor: No necesitas aprovisionar ni administrar servidores. Lambda se encarga de toda la infraestructura subyacente.

Escalado automático: Lambda escala automáticamente en función del número de solicitudes, permitiendo manejar desde unas pocas hasta miles por segundo.

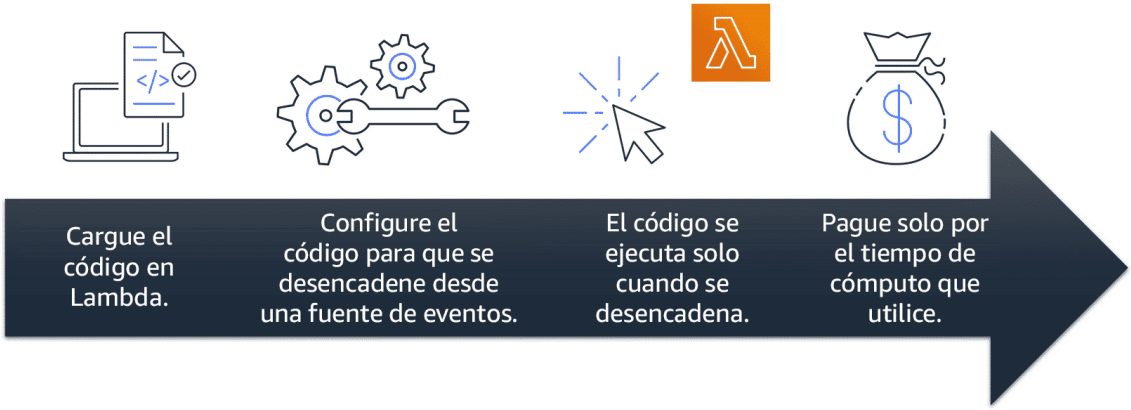
Pago por uso: Solo pagas por el tiempo de computación que consumes, sin cargos cuando el código no se está ejecutando.

Integración con otros servicios de AWS: Lambda puede integrarse fácilmente con otros servicios de AWS como S3, DynamoDB, Kinesis, entre otros.

Aplicaciones basadas en eventos: Lambda ejecuta el código en respuesta a eventos como cambios en datos, solicitudes HTTP, y más.

Alta disponibilidad: Lambda ejecuta el código en una infraestructura de alta disponibilidad, gestionando la capacidad, el mantenimiento del servidor y el sistema operativo

FUNCIONAMIENTO



1. Suba el código a Lambda.

2. Establezca el código para que se active desde una fuente de eventos, como servicios de AWS, aplicaciones móviles o puntos de enlace HTTP.

3. Lambda ejecuta su código solo cuando se activa.

4. Solo paga por el tiempo de cómputo que utiliza. En el ejemplo anterior de cambio de tamaño de imágenes, pagaría solo por el tiempo de cómputo que utiliza cuando se cargan nuevas imágenes. Al cargar las imágenes, Lambda ejecuta el código para la función de cambio de tamaño de imagen.

CUANDO USAR LAMBDA

Procesamiento de archivos: Utilice Amazon Simple Storage Service (Amazon S3) para activar el procesamiento de datos de Lambda en tiempo real después de una carga.

Procesamiento de transmisiones: Utilice Lambda y Amazon Kinesis para procesar datos de streaming en tiempo real para el seguimiento de la actividad de las aplicaciones, el procesamiento de pedidos de transacciones, el análisis de secuencias de clics, la limpieza de datos, el filtrado de registros, la indexación, el análisis de redes sociales, la telemetría de datos de dispositivos de Internet de las cosas (IoT) y la medición.

Aplicaciones web: Combine Lambda con otros servicios de AWS para crear potentes aplicaciones web que se amplíen y reduzcan verticalmente automáticamente y se ejecuten en una configuración de alta disponibilidad en varios centros de datos.

Backends de IoT: Cree backends sin servidor con Lambda para gestionar solicitudes de API web, móviles, IoT y de terceros.

Backends móviles: Cree backends con Lambda y Amazon API Gateway para autenticar y procesar solicitudes de API. Utilice AWS Amplify para integrarse fácilmente con sus frontends de iOS, Android, Web y React Native.

BENEFICIOS

No es necesario administrar servidores: Ejecute el código sin aprovisionar ni administrar la infraestructura. Simplemente escriba y cargue el código como un archivo .zip o una imagen de contenedor.

Escalamiento automático: Responda automáticamente a las solicitudes de ejecución de código a cualquier escala, desde una docena de eventos al día hasta cientos de miles por segundo.

Precio de pago por uso: Ahorre costos al pagar solamente por el tiempo de computación que utiliza, por milisegundo, en lugar de aprovisionar la infraestructura por adelantado para la capacidad máxima.

Optimización del desempeño: Optimice el tiempo de ejecución del código y el rendimiento con el tamaño adecuado de la memoria de las funciones. Responda a la alta demanda en milisegundos de dos dígitos con simultaneidad aprovisionada.

CASOS DE USO

Procese datos rápidamente a escala: Satisfaga una demanda impredecible y que requiere de muchos recursos usando AWS Lambda para escalar horizontalmente al instante a más de 18 000 vCPU. Cree flujos de trabajo de procesamiento de forma rápida y sencilla con un conjunto de otras ofertas sin servidor y desencadenadores de eventos.

Ejecute backends web y móviles interactivos: Combine AWS Lambda con otros servicios de AWS para crear experiencias en línea seguras, estables y escalables.

Habilite ML Insights para un potente monitoreo de datos: Preprocese los datos antes de alimentar su modelo de machine learning (ML). Con el acceso a Amazon Elastic File System (EFS), AWS Lambda se encarga de la administración y el aprovisionamiento de la infraestructura para simplificar el escalado.

Cree aplicaciones basadas en eventos: Cree funciones basadas en eventos para facilitar la comunicación entre servicios desacoplados. Reduzca los costos al ejecutar aplicaciones durante los momentos de máxima demanda sin que se produzcan colapsos o un exceso de aprovisionamiento de recursos.

ANATOMÍA DE UNA FUNCIÓN LAMBDA

RESTRICCIONES DE LAS FUNCIONES LAMBDA

1. **Límites de recursos:**

- Asignación de memoria: Puede asignar memoria entre 128 MB y 10.240 MB (10 GB) en incrementos de 1 MB.
- Tiempo de espera máximo de ejecución: El tiempo máximo de ejecución de una función de Lambda es de 15 minutos. Si la función supera este límite, se finaliza.
- Almacenamiento efímero: Cada función de Lambda obtiene 512 MB de espacio en disco temporal en el /tmp directorio para almacenar archivos.
- Límite de simultaneidad: El límite de simultaneidad predeterminado por cuenta es de 1.000 invocaciones simultáneas (esto se puede aumentar a través de una solicitud de soporte técnico). También puede establecer simultaneidad reservada para funciones individuales.
- Variables de entorno: Se pueden configurar hasta 4 KB de variables de entorno.

2. **Límites de invocación:**

- Tamaño de la carga útil:
  En el caso de las invocaciones sincrónicas, el límite de tamaño de carga es de 6 MB.
  En el caso de las invocaciones asincrónicas, el límite de tamaño de carga es de 256 KB.
- Tamaño de solicitud/respuesta: El tamaño de la carga útil de solicitud y respuesta tiene un límite de 6 MB para las solicitudes sincrónicas.

3. **Tamaño del paquete de implementación:**

- Carga directa: El límite de tamaño para el paquete de implementación de una función cuando se carga a través de la consola es de 50 MB (archivo ZIP comprimido).
- Carga de S3: Cuando se utiliza S3 para cargar un paquete de implementación, el límite de tamaño es de 250 MB (archivo ZIP comprimido).
- Tamaño sin comprimir: El tamaño del paquete de implementación sin comprimir (una vez cargado en la memoria) puede ser de hasta 250 MB.

4. **Entorno de ejecución:**

- Restricciones de tiempo de ejecución: AWS Lambda admite tiempos de ejecución específicos (por ejemplo, Node.js, Python, Java, etc.). El uso de otros lenguajes requiere el uso de un tiempo de ejecución personalizado.
- Sin estado persistente: Lambda no tiene estado. No hay garantía de que se ejecuten ejecuciones consecutivas en la misma instancia, por lo que cualquier estado o archivo almacenado localmente se perderá entre invocaciones.
- Tiempo de espera de ejecución: Una función de Lambda se limita a un tiempo de ejecución de 15 minutos antes de que se termine a la fuerza si aún se está ejecutando.

EJEMPLOS DE FUNCIONES LAMBDA

Estos son unos ejemplos básicos, pero las funciones Lambda pueden ser mucho más complejas y realizar una variedad de tareas.

- Función Lambda que lista los buckets S3 en una cuenta de AWS: Para esta práctica previamente debemos tener creado mínimo un bucket S3 en nuestra cuenta de AWS.

```
import json
import boto3

def lambda_handler(event, context):
    # Listar buckets S3
    print('Hola mundo desde lambda')
    client = boto3.client('s3')
    response = client.list_buckets(
        MaxBuckets=10
    )
    return json.dumps(response, default=str)
```

Descripción del código:

import json: Se utiliza para importar el módulo JSON de Python. Este módulo permite trabajar con datos en formato JSON (JavaScript Object Notation), que es un formato común para intercambiar datos entre un cliente y un servidor.

import boto3: Se utiliza para importar la biblioteca Boto3, que es el SDK (Software Development Kit) de AWS para Python. Esta biblioteca permite a la función Lambda interactuar con los servicios de AWS, como S3, EC2, DynamoDB, entre otros.

def lambda_handler(event, context): Es el nombre comúnmente utilizado para la función principal que AWS Lambda invoca cuando se ejecuta el código. Esta función actúa como el punto de entrada para el código Lambda y se define en el archivo de código que subimos a AWS Lambda.
Se compone de un evento y un contexto: El evento contiene toda la información sobre el evento que desencadenó esta Lambda y el contexto contiene información sobre el entorno donde se está ejecutando nuestra función Lambda.

 # Listar buckets S3: Contiene un comentario con la descripción de lo que hace la función Lambda, los comentarios inician con numeral (#).

print('Hola mundo desde lambda'): El mensaje que se imprime en pantalla.

client = boto3.client('s3'): Declara un objeto cliente.
El objeto cliente se crea a partir de la librería boto3, en este caso el cliente utiliza el servicio S3, pero también puede usar SES, EKS, o cualquier otro servicio de AWS.
Una vez se declara el cliente, queda listo para utilizarse como un solo servicio.

response = client.list_buckets: La respuesta, en este caso es igual a la lista de buckets que el cliente S3 va a obtener.

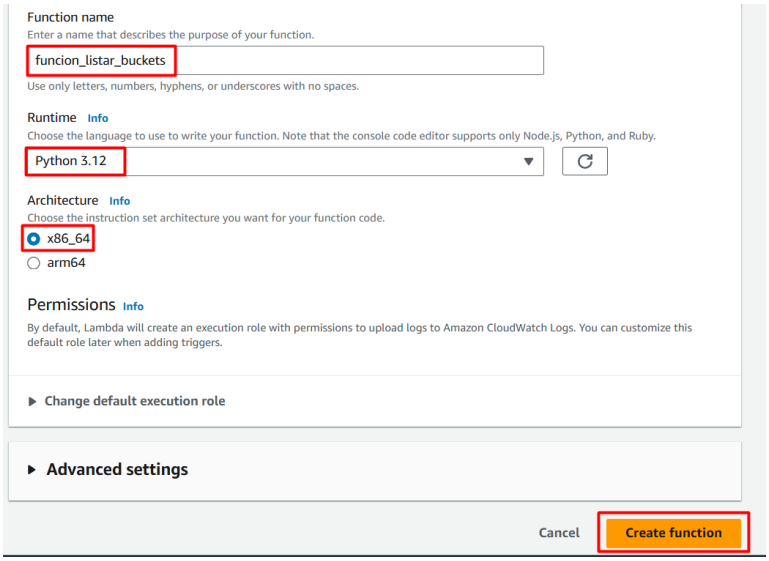
MaxBuckets=10: Definimos que máximo nos devuelva 10 buckets de S3.

return json.dumps(response, default=str): Lo que devuelve la función lambda.
El comando return json.dumps(response, default=str) en una función Lambda convierte un objeto de Python en una cadena JSON. El parámetro default=str se utiliza para manejar objetos que no son serializables por defecto, como los objetos de fecha y hora. Esto asegura que cualquier objeto que no pueda ser convertido directamente a JSON se convierta a una cadena en su lugar.

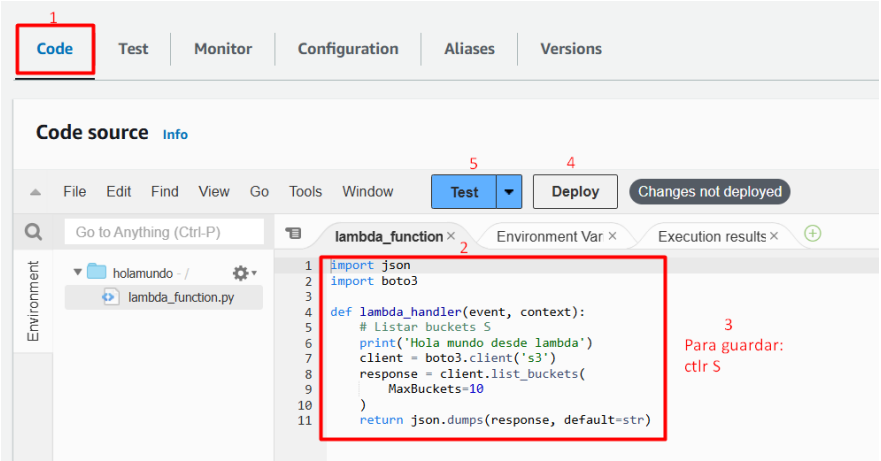
PERMISOS: El rol de ejecución de esta lamba debe tener permiso de listar buckets sobre el servicio S3.

Pasos:

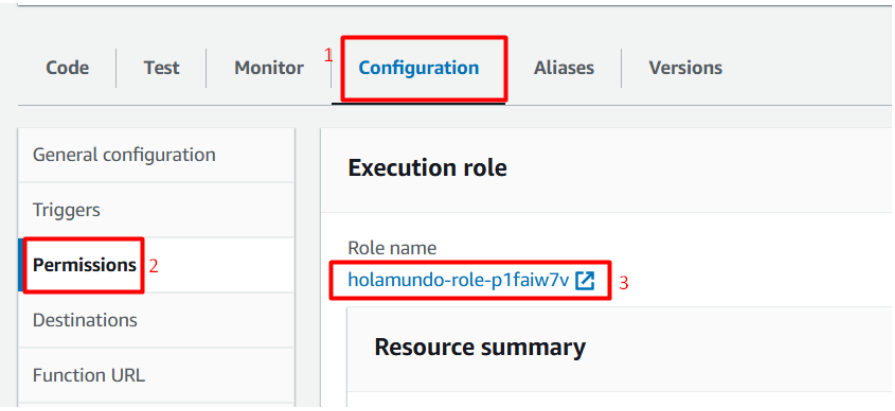
1. Ingresamos al servicio AWS Lambda, damos clic en Crear función, le asignamos un nombre a la función, seleccionamos el Runtime Python, dejamos las demás configuraciones por defecto y damos clic en Create function:



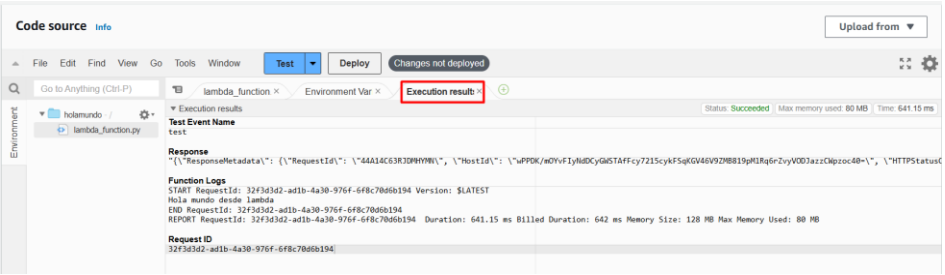
2. Pegamos nuestro código, guardamos con ctrl S, damos clic en Deploy, desplegamos el menú de Test, seleccionamos Test y damos clic en el botón azul Test:



3. Es muy probable que falle por permisos, se debe asignar el permiso para listar buckets al rol de la función lambda sobre el servicio S3:

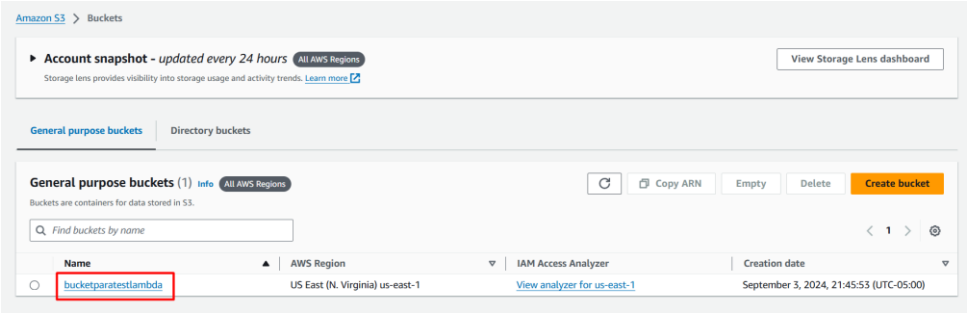


4. La correcta ejecución de la función Lambda debe devolvernos un resultado como este:



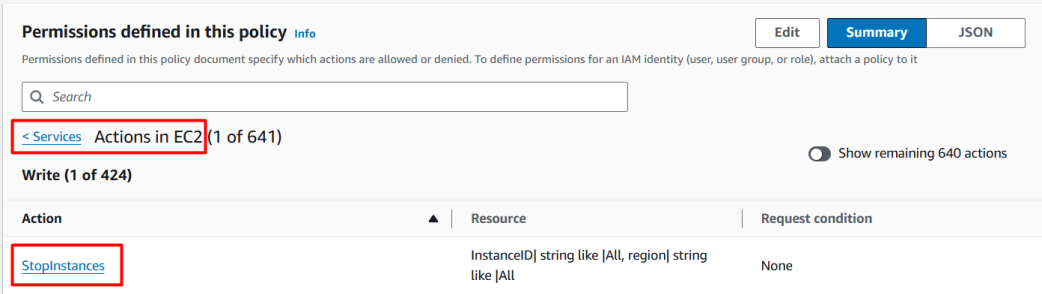
Al mover la barra de desplazamiento horizontal debemos ver el bucket listado:

Debe coincidir con el bucket en la consola de S3:



- Función Lambda para apagar una instancia EC2: Para esta práctica debemos tener previamente creada una instancia EC2 en estado running o en ejecución.

PERMISOS: El rol de ejecución de esta lamba debe tener permiso de apagar instancias sobre el servicio EC2:

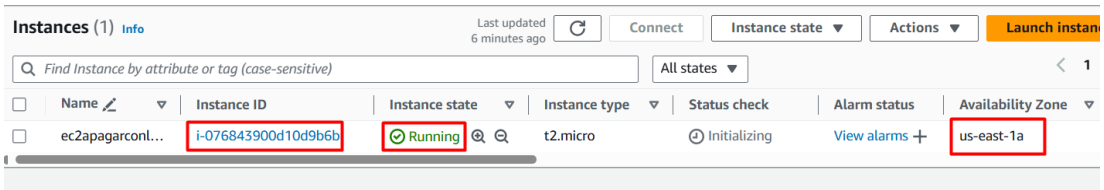


Import os: Se utiliza para importar el módulo os de Python, que proporciona una forma de interactuar con el sistema operativo. Este módulo permite realizar diversas operaciones, como manipulación de archivos y directorios, obtención de información del sistema, y ejecución de comandos del sistema.

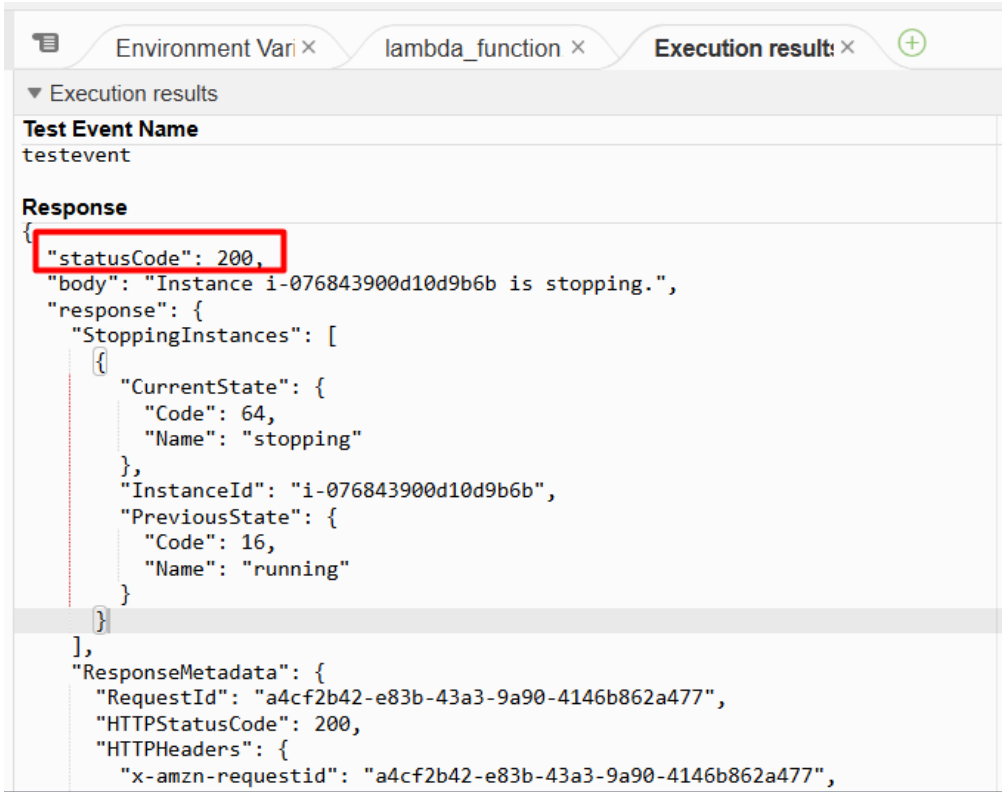
```
import json
import boto3
import os

# Crear un cliente de EC2 usando boto3
ec2_client = boto3.client('ec2', region_name=os.getenv('AWS_REGION', 'us-east-1'))

def lambda_handler(event, context):
    # ID de la instancia que se desea iniciar o detener
    instance_id = 'i-076843900d10d9b6b'

    # Determinar la acción a tomar ('start' o 'stop')
    action = event.get('action', 'stop')  # 'stop' es el valor por defecto

    if action == 'start':
        response = start_instance(instance_id)
        return {
            'statusCode': 200,
            'body': f'Instance {instance_id} is starting.',
            'response': response
        }
    elif action == 'stop':
        response = stop_instance(instance_id)
        return {
            'statusCode': 200,
            'body': f'Instance {instance_id} is stopping.',
            'response': response
        }
    else:
        return {
            'statusCode': 400,
            'body': 'Invalid action. Use "start" or "stop".'
        }

def start_instance(instance_id):
    try:
        response = ec2_client.start_instances(InstanceIds=[instance_id])
        return response
    except Exception as e:
        return str(e)

def stop_instance(instance_id):
    try:
        response = ec2_client.stop_instances(InstanceIds=[instance_id])
```

```
    return response
except Exception as e:
    return str(e)
```

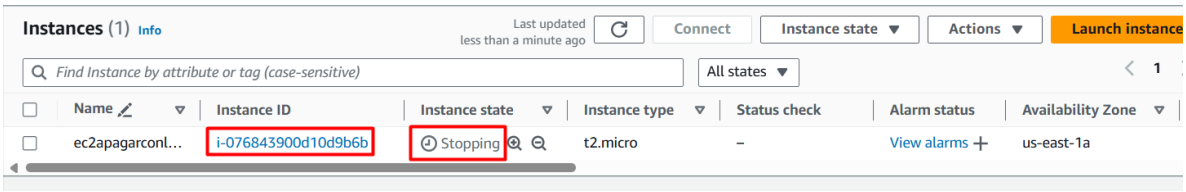
IMPORTANTE: Reemplazar los valores en rojo por la región de su cuenta y el ID de su instancia EC2.



Luego de ejecutar la función Lambda debemos obtener un statusCode: 200, que indica que la operación fue exitosa:



Confirmamos en la consola de Amazon EC2 que la instancia se apagó correctamente, desde la función Lambda:



TIPS

- Para guardar el código de la función Lambda: Ctrl S -> Deploy -> Test -> Save.
- Si elijo Python como runtime debo escribir todo en minúscula y separado por guion bajo.
- Tener cuidado con las identaciones (espacios): Después de lambda_handler se identa 4 espacios, después de dos puntos (:) se identa y después de paréntesis también se identa.
- Siempre idento 4 espacios, es espaciar, NO tabular.
- Se sugiere, para empezar a trabajar con funciones Lambda, usar Python porque es más fácil de usar que Node.js, sin embargo, la decisión también depende del lenguaje de programación con el que trabaje el equipo de desarrollo.